

Stack ADT, Operations & Array Implementation

Q. Define Stack as an Abstract Data Type (ADT). Explain its primary operations and demonstrate how a Stack can be implemented using an Array with suitable code snippets.

> **Definition to Remember**

> A **Stack** is a linear data structure that follows the Last In First Out (LIFO) principle. It supports the following operations:
- push: Add an element to the top of the stack.
- pop: Remove an element from the top of the stack.
- empty: Check if the stack is empty.

1. Stack as an ADT

Operation	Description	Condition
-----------	-------------	-----------

| :--- | :---

2. Stack Diagram

30 top

Initial (0)
20

3. Array Implementation in C

```
```c
```

```
#include <stdio.h>
```

```
int stack[MAX];
```

```
int top = -1;
```

```
void push(int value) {
```

```
if (top == MAX - 1) {
```

```
int pop() {
```

```
if (top < 0) {
```

```
int peek() {
```

```
if (top
```

```
int isEmpty() { return top == -1; }
```

```
...
```

#### 4. Advantages and Limitations

```
| | Array Implementation |
```

```
| :--- | :
```

```

```

```
> **Must-Write Points (for 10 marks)**
```

```
> 1. St
```

```

```

```
> **Quick Recall**
```

```
> `Stac
```

```

```

### Complete Executable C Code: Array Implementation of Stack ADT

```
```c
```

```
#includ
```

```
#define MAX 5
```

```
struct Stack {
```

```
int item
```

```
void initStack(struct Stack *s) {
```

```
s->top
```

```
int isFull(struct Stack *s) {
```

```
return
```

```
int isEmpty(struct Stack *s) {
```

```
return
```

```
void push(struct Stack *s, int val) {
```

```
if (isFu
```

```
int pop(struct Stack *s) {
```

```
if (isEr
```

```
int peek(struct Stack *s) {
```

```
if (isEr
```

```
void display(struct Stack *s) {
```

```
if (isEr
```

```
int main() {
```

```
struct s
```

```
push(&s, 10);
```

```
push(&
```

```
printf("Popped Value: %d\n", pop(&s));
```

```
display
```

```
printf("Top Element (Peek): %d\n", peek(&s));
```

return